CO226-Apr2025 : Database Systems

Lab 07 : Django + MySQL

<u>Part 1 : Connecting to an existing database</u>

In this part of the lab, we will be using the database from the previous lab (Lab06) so make sure you already have the database named “university” and it has students, courses, enrollments, instructors, course_instructors tables and is populated. (if not please run the sql script that was given to you in the lab06 folder).

Now lets get started with Django,

- Step 1 – install Django
  (we assume you have installed python, if not please install python)
  - Please install django and mysql client for python by running the following command
    pip install django mysqlclient
- Step 2 – create a project
  -
    ```
    django-admin startproject university_backend
    cd university_backend
    ```
    That will create,
    ```
    university_backend/
    ├── manage.py
    └── university_backend/
        ├── __init__.py
        ├── settings.py
        ├── urls.py
        └── wsgi.py
    ```
  - manage.py – CLI for managing your project.
  - settings.py – Configurations for the database and other stuff
- Step 3 – config the database
  - Open university_backend/settings.py and **replace** the default DATABASES section with,

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'university',
        'USER': 'your mysql username (root)',
        'PASSWORD': 'your_mysql_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

- Step 4 – create django app

  python manage.py startapp university

  ```
  university_backend/
  ├── university/          ← your app that you just created
  │   ├── models.py
  │   ├── views.py
  │   ├── urls.py   ← lets create this file later
  │   └── ...
  ```

  - In settings.py (within the university_backend folder, **add** your app to INSTALLED_APPS:

```
INSTALLED_APPS = [
    ...
    'university',
]
```

- Step 5 – define models

Here, if the database exist we can automatically create the model by the command
python manage.py inspectdb > university/models.py
but let's skip that and do it manually

  - Here you have to define the models in a way that matches the existing schema
  - Edit university/models.py to match the MySQL schema:

```python
from django.db import models

class Student(models.Model):
    student_id = models.IntegerField(primary_key=True)
    name = models.CharField(max_length=100)
    dob = models.DateField()
    email = models.CharField(max_length=100)
    gpa = models.DecimalField(max_digits=3, decimal_places=2, null=True)

    class Meta:
        db_table = 'students'

class Course(models.Model):
    course_id = models.IntegerField(primary_key=True)
    course_name = models.CharField(max_length=100)
    credits = models.IntegerField()

    class Meta:
        db_table = 'courses'

class Instructor(models.Model):
    # TODO

class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    enroll_date = models.DateField()
    grade = models.CharField(max_length=1, null=True)

    class Meta:
        db_table = 'enrollments'
        unique_together = ('student', 'course')

class CourseInstructor(models.Model):
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    instructor = models.ForeignKey(Instructor, on_delete=models.CASCADE)

    class Meta:
        db_table = 'course_instructors'
        unique_together = ('course', 'instructor')
```

- Step 6 – tell django to use existing table
    - Run
      python manage.py makemigrations university
- Step 7 – create a simple view
    - Edit university/views.py to create a simple endpoint

```
from django.http import JsonResponse
from .models import Student

def student_list(request):
    students = list(Student.objects.values())
    return JsonResponse(students, safe=False)

#TODO : later create more methods to view other stuff
```

- Step 8 – wire up URLs
    - Create a new file: university/urls.py (which will contail urls for the app)

```
from django.urls import path
from . import views

urlpatterns = [
    path('students/', views.student_list),
]
```

    - Now in In university_backend/urls.py, include your app's URLs

```
from django.contrib import admin
from django.urls import path, include # add this include here

urlpatterns = [
    path('admin/', admin.site.urls),
    path('unidata/', include('university.urls')),
]
```

- Step 9 – run the server
  python manage.py runserver

now go to
http://127.0.0.1:8000/unidata/students/

Lab Tasks

1. Now create a view to get the courses data and wire up the urls properly and get the data to be viewed from a web browser. Take screenshots.

Now lets see how to insert data from the admin terminal of the django

- Step 10 – creat a super user
  python manage.py createsuperuser
  Here you will be asked to give a username, email and a password
  - In university/admin.py:

```
from django.contrib import admin
from .models import Student, Course, Enrollment, Instructor,
CourseInstructor

admin.site.register(Student)
admin.site.register(Course)
admin.site.register(Enrollment)
admin.site.register(Instructor)
admin.site.register(CourseInstructor)
```

Now go to,
http://127.0.0.1:8000/admin/

Here you can login and manage the database.

Lab Tasks

2. Add new data to students and courses tables from the admin terminal and view it from the previously created views and urls – view on browser. Take screenshots.

Part 2 : Using a new database

Now lets create a **new project** that can be used as a bankend for a bank.

- Step 1 – Set Up Project
  django-admin startproject banking_backend
  cd banking_backend
- Step 2 – create app
  python manage.py startapp bank
- Step 3 –
  Create a database named "banking_system" in you mysql server manually
- Step 4 – Register it in banking_backend/settings.py
  INSTALLED_APPS = [
  ...
  'bank',
  ]

  and then modify the banking_backend/settings.py to connect database

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'banking_system',
        'USER': username(root?),
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

- Step 5 – Define the models in bank/models.py

```
from django.db import models

class Customer(models.Model):
    name = models.CharField(max_length=100)
    dob = models.DateField()
    email = models.EmailField(unique=True)
    address = models.TextField()

    def __str__(self):
        return self.name
```

```python
class Account(models.Model):
    ACCOUNT_TYPES = [
        ('SAV', 'Savings'),
        ('CUR', 'Current'),
    ]

    customer = models.ForeignKey(Customer, on_delete=models.CASCADE)
    account_type = models.CharField(max_length=3, choices=ACCOUNT_TYPES)
    balance = models.DecimalField(max_digits=12, decimal_places=2)
    created_at = models.DateField(auto_now_add=True)

    def __str__(self):
        return f"{self.customer.name} - {self.account_type}"

class Transaction(models.Model):
    account = models.ForeignKey(Account, on_delete=models.CASCADE)
    TRANSACTION_TYPES = [
        ('D', 'Deposit'),
        ('W', 'Withdrawal'),
    ]
    transaction_type = models.CharField(max_length=1,
choices=TRANSACTION_TYPES)
    amount = models.DecimalField(max_digits=10, decimal_places=2)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.get_transaction_type_display()} of {self.amount} on
{self.timestamp}"
```

- Step 6 – Create the Database and Tables
  python manage.py makemigrations
  python manage.py migrate
  - This will create the tables
- Step 7 — Add Data via Admin
  - Create Superuser
    python manage.py createsuperuser
  - Register Models in bank/admin.py

```
from django.contrib import admin
from .models import Customer, Account, Transaction

admin.site.register(Customer)
admin.site.register(Account)
admin.site.register(Transaction)
```

- Step 8 – run server
  python manage.py runserver
- Step 9 – use GUI and add some data to the database
  http://127.0.0.1:8000/admin/
- Step 10 – View the Data via an API
  - Add a basic view in bank/views.py:

```
from django.http import JsonResponse
from .models import Customer

def customer_list(request):
    data = list(Customer.objects.values())
    return JsonResponse(data, safe=False)
```

Then in bank/urls.py (create the file if needed)

```
from django.urls import path
from . import views

urlpatterns = [
    path('customers/', views.customer_list),
]
```

Link to it in the main urls.py:

```
from django.urls import path, include

urlpatterns = [
    path('users/', include('bank.urls')),
    path('admin/', admin.site.urls),
]
```

- Step 11 – view data,
  http://127.0.0.1:8000/users/customers/

Lab Tasks

3. Take screenshots of the outcome of step 11 on the browser.

Create a pdf using the screenshots of all three lab tasks, rename it as E21XXX_CO226_Lab07.pdf and submit.